

# PARSE2VEC: TRANSFORMER EMBEDDINGS DEEPER THAN SURFACE FORMS

MICHAEL CONNOR

**ABSTRACT.** **Objective:** The minimum transformer performance can move forward by incorporating the progress of the last decade. **Background:** Current efforts begin with surface forms and build meanings for tagging, parsing, identifying semantic relationships, etc. But a parser and outside resources can also provide these morphological, syntactic, and semantic features. **Method:** Instead of inputting tokens of surface forms into a transformer, we could start with feature vectors whose summed embeddings represent the deep features of the words, changing it to structured data. This is an empirical study on how inputs with feature vectors perform on masked word prediction. Compared to classic transformers, increasing integration between the parser and transformer accumulates to a total accuracy gain of 143% and perplexity loss of 87%.

## Background

The latent information in a transformer model and in word embeddings covers a diverse field over more than a decade. The impetus for this work is from (16, 14, 15), who cataloged syntactic and semantic word relations, and showed how words are composable through their relationships. In (3), surface form words, root words, suffixes, and parts of speech form various feature vectors for word embedding reasoning tasks. In (12), surface forms and dependencies from spaCy combine as complex values for translations. In (7), spaCy output including sub-word tokens and dependencies create a gloss representation for translations. In (31), they showcased the internal model's ability to perform parsing related tasks while trained for masked word prediction. In (10) and (2), sub-word segments combine from byte pair encoding plus a morphological segmentation table, in order to produce better sub-word tokenization. In (8), named entity recognition marking boosts the identification of legal clauses and metadata. In (4), gendering in words introduces bias in word embeddings. In (21), entities and coreferences combine within the transformer during training. In (18), the inclusion and removal of seemingly superfluous tokens like punctuation proves consequential. In (22), a transformer module includes a clause classification stage in order to prioritize and finally classify sentiment. In (24), among other findings, related absolute positions were found to be equivalent but preferred to learned absolute positions. In (20), relative positions outperform absolute positions in some tests. In (26), radial basis function kernels act as activation functions within a neural network. And the Transformer of (24) uses surface forms, but can easily accommodate multimodal input.

All of these seem unrelated, but all of these involve latent information within the transformer model and a parser can output features of all of these and attach outside resources for others. Given the conclusions of past work that transformer models contain these relationships from surface form text, a transformer model with these relationships as input should have comparable results.

## Method

The training corpus goes through the spaCy (9) parser and combines with outside csv tables. That parser output becomes the input for a modified transformer model. The transformer model is an encoder that predicts words.

Among other parsers tested, spaCy has a free and publicly available version with ample documentation. For this paper, all resources, scripts, sample data, and source code can be found on GitHub<sup>1</sup>.

Each input word is represented by a feature vector rather than by the surface form alone as a token. A transformer using surface form input and with surface form output for masked word prediction marks the baseline for comparison. The tests cover the gradual addition of features in groups, with an individual focus on Stanford Dependencies. The transformer model resembles a multimodal model with early fusion. The baseline uses fusion by addition as in (24), which is compared to other common fusion functions. Activation functions produce features which interact as values within the standard transformer architecture without changing its explicit activation functions. A sparsity function operates in  $O(nm)$  complexity by essentially truncating the key and value in a sparsity attention step from  $n$  to  $m$ , and relying on the features of the parser to fill in the difference. Masked word prediction relates directly to the model training and reflects a basic function of a transformer model. The metrics for the experiment are accuracy and perplexity of predicted words.

**Configuration** The scale of this experiment fits within the constraints of the BabyLM challenge (6). The volume of data in training is 700K words with 7700 unique words and in testing it is 250K words. This data comes from the Newsgroups20 corpus (11) for training and the AG News corpus (29) for testing. The experiments run on a MSI GF75 Thin laptop with a Nvidia RTX 3060. The transformer architecture settings uses a combination of minimal settings for BabyLM constraints: sequence length = 16, attention heads = 4, layers = 3, hidden dimension = 128, feed forward dimension = 512; and default settings from (23): feed forward Dropout = 0.1, embedding Dropout = 0.5, embedding LayerNorm epsilon = 1e-12, encoder LayerNorm epsilon = 1e-5, optimizer = Adam with default settings.

**Preprocessing** SpaCy produces *token* objects containing *text*, *tag\_* which are Penn tags (19), *pos\_* which are Universal Part Of Speech tags (17), *lemma\_*, *dep\_* which are Universal Stanford Dependency tags (13), *morph*, and *ent\_type\_* from *en\_core\_web\_lg*. A second parser instance runs to produce *coref\_clusters\_* tags from *en\_coreference\_web\_trf*. Based on these, head dependency tags, inventory tags, lemma pos tags, dependency pos tags, folded tags, and more position features are created. Finally, outside csv tables are loaded to add root words, suffixes, and sub-suffixes.

There is a 5 word minimum frequency threshold, below which words are replaced with *UNK*. All digits are converted to *NUM*. Numbers that appear spelled out remain. Except for folding, all punctuation is dropped after spaCy uses it for tokenization. All embeddings for all features start with Xavier initialization.

For feature vectors, minimum thresholding eliminates features but not entire words. Features of entries are not removed but replaced with *UNK*. Since other features provide a partial view, these are comparable to other incomplete entries like numbers whose numeric values are replaced with *NUM*.

Features like the elements of spaCy’s *token* appear multiple times in previous work, often in pairings like the surface form and dependency couples (7, 12), and the elements of *token* fill out

<sup>1</sup><https://github.com/maconnor34/parse2vec>

the familiar elements of the feature vector. The new elements begin with a head dependency tag, which is a copy of the dependency tag attached to the head of the compliment or modifier and marked as head. For example in "brown fox", *amod* attaches to "brown" and *head\_amod* attaches to "fox". The data from *ent\_type\_* is included for its semantic categories as in (8), and also for its syntactic fit in the sentence. Root words, suffixes and sub-suffixes are not sub-word tokens and are not defined by byte pair encoding like in (10). They are copied from a morphological segmentation table with major suffix combinations. Gender features come from a list of common gendered nouns in order to flag obvious gendering but not debias it as in (4). Incorporating coreferences happens here before the transformer and derives from the practice of (21), but instead of replacing single tokens, lemma tags of coreferences copy to pronouns and gender tags of pronouns copy to coreferences. An inventory tag acts as a clause classification of (22), and marks pos tags missing from the current phrase, for example *missing\_NOUN* and attaches to the *MASK* feature vector. A lemma pos tag combines the lemma tag with the pos tag, for example, *lemma\_fox\_NOUN*. A dependency pos tag combines the dependency tag with the pos tags of its modifier and head, for example, *amod\_ADJ\_NOUN*. These combination tags act as subcategories of their constituents and as a pivot to learning about their constituent tags. There is no normalizing for any features as all features have random initialization. Though many features have sequential, associative, or hierarchical relationships; all relationships within the model are learned.

Multiple position features explicitly represent the complex relationships of word order. There are six positioning dimensions present: learned absolute positions, relative positions, relative positions within sets, parse tree position from dependency tags, dependency pointers, and repeated word pointers. The absolute position token moves from its own set of tokens into the main feature set. Then there are relative positions like (20). Combining parser output with positions produces relative positions within sets of parts of speech such as *noun-2, noun-1, noun+1, noun+2* to indicate the previous or subsequent nouns. In addition to their qualitative meanings, the dependency tags and their head dependency tags mark relative positions within parse trees. Since a modifier dependency tag links to a corresponding head tag but does not point to it, dependency pointer tags explicitly point to the absolute position but do not link to a specific dependency. Repeated word pointers point to the absolute position of another occurrence of the same word.

Fold tags differ substantially from the spaCy tags in their organization. Fold tags echo another tag from words conforming to constraints and also offer the ability to add punctuation information without adding words to the sequence. The fold tag itself copies the lemma pos tag and appears as *fold\_lemma\_fox\_NOUN*. The fold tags come from *UNK, NUM*, punctuation, symbols, x, interjections, particles, determiners, pronouns, auxiliaries, prepositions, adjectives, adverbs, conjunctions, direct objects, indirect objects, objects of prepositions, compounds, and verbs. Some of these are universal tags and some are dependencies. They all fold onto every word that they modify from any dependency tag.

The feature vectors represent context words as in (3). These features have exclusive subsets, *NONE* features, and a hard vector length. All the features of these vectors require 5 times the original set of features for surface forms. The number of features for root words, lemmas, lemma pos tags, and fold tags scale with the number of surface forms. There are less than 1000 features for all other metadata. Target words are restricted to surface forms. The dimensions of the input far exceed that of the output.

Creating a view of the parser output that masks the target word requires multiple parser passes beyond the unmasked baseline. A gibberish word replaces every 12th word to simulate

it being unknown, and therefore 12 sets of modification annotations attach to every feature vector. When marshalling the data for processing, the feature vectors of every context word lose pieces as if the masked word was the 12th unknown word. Dependencies disappeared most commonly and at the greatest distance.

**Processing** The classifier is a classic transformer encoder with source code from (23), a straight forward implementation of (24). A 44 dimension input replaces the 1 dimensional surface form with a 1 dimensional position. The fusion function occurs in the call to Embedding.

$$\begin{aligned} x &= e_{surface\_form} + e_{position} \\ Q &= xq, K = xk, V = xv \\ Attention(Q, K, V) &= softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V \end{aligned}$$

This is the classic method for passing the input into the transformer model. In (24), the *surface\_form* word tokens and *position* tokens rest in two separate Embedding objects with different initializations. The embedding passes to 3 projection matrices to form  $Q$ ,  $K$ , and  $V$ . Finally the softmax function activates the dot product of  $Q$  and  $K$ , and then projects back into  $V$  to get an attention vector to add to the embedding vector.

$$x = \sum_{i=1}^{d_e} e_i$$

This calculation of  $x$  extends (24), the sum of embeddings of the dimensions of  $e$  including the *surface\_form* and *position*, and all features initialize together. The multimodal features come from an external source, a parser, but they stay within the internal meaning of the word, which the Embedding object presents to the rest of the model. This changes the lookup process for an embedding, and this changes the flow of learning within the model. The features do not hide the internal dimensions of single word representations, but rather name them and demarcate them. Before, words that were nouns each had an unnamed noun dimension related to each noun and those dimensions had common unnamed noun dimensions. With feature vectors, noun words have unnamed noun dimensions that relate to each word, but their common noun dimension has a feature name. In the classic transformer of (24), the embeddings of surface form tokens begin as words and then relationships flow backward from predictions into feed forward networks into attention heads of the model and into the word embeddings during training. Word feature vectors provide words and relationships, so that these internal representations flow in both directions.

Other established multimodal early fusion algorithms are tested as well.

$$x = norm(W)e$$

This calculation of  $x$  uses a vector of normalized weights  $W$  of dimension  $1 \times d_e$  to produce a weighted sum by matrix multiplication, as in (25).

$$\begin{aligned} g &= concat(e_1, \dots, e_d) \\ x &= gW \end{aligned}$$

This calculation of  $x$  uses a concatenation of the embeddings of  $e$  and a projection matrix  $W$  of dimension  $d_e * hidden\ dimension \times hidden\ dimension$ , also described in (25).

$$\begin{aligned} h &= \tanh(eW) \\ k &= \text{concat}(h_1, \dots, h_d) \\ z &= \text{sigmoid}(Vk) \\ x &= \sum hz \end{aligned}$$

This calculation of  $x$  uses a gated multimodal unit from (1).  $W$  and  $V$  are learned weights vectors of dimensions  $\text{hidden dimension} \times d_e$  and  $d_e \times d_e \times d_e$ .

$$\begin{aligned} h &= \tanh(eW) \\ k &= \sum_{i=1}^{d_e} h_i \\ z &= \text{sigmoid}(Vk) \\ x &= zh \end{aligned}$$

This calculation of  $x$  uses a gated fusion function based on a sum instead of concatenation.  $W$  and  $V$  are learned vectors of weights of dimension  $\text{hidden dimension} \times \text{hidden dimension}$ .

The following kernel methods focus on interaction, based off of the sum of embeddings. The radial basis function provided the impetus to assay multiple kernel functions.

$$\begin{aligned} RBF(e_{\text{root word}}, e) &= \sum_{i=1}^{d_e} \exp(-\gamma \|e_{\text{root word}} - e_i\|^2) \\ x &= \sum_{i=1}^{d_e} e_i + RBF(e_{\text{root word}}, e)W \end{aligned}$$

This calculation is a radial basis function centered around one feature to add a distinct feature specifically representing how the features interact together. The radial basis function is adapted from the goal of (26), but instead of being an activation function it fits into the feature space as an activation feature for the feed forward step to use. A projection matrix  $W$  aligns its dimensions with  $x$ .

$$\text{kernel}_e = e_{\text{root word}} \cdot e^T$$

The implementation of the radial basis function uses this kernel in calculating  $\|e_{\text{root word}} - e_i\|^2$ . All of the following kernel functions share this same kernel.

$$\begin{aligned} \cos(e) &= \text{kernel}_e / (\text{norm}(e_{\text{root word}}) * \text{norm}(e)) \\ x &= \sum_{i=1}^{d_e} e_i + \cos(e)W \end{aligned}$$

This calculation of  $x$  uses cosine similarity as a kernel function to create an interaction feature. A projection matrix  $W$  aligns its dimensions with  $x$ .

$$x = \sum_{i=1}^{d_e} e_i + \tanh(\text{kernel}_e)W$$

This calculation of  $x$  applies the hyperbolic tangent function to the kernel to produce an interaction feature. A projection matrix  $W$  aligns its dimensions with  $x$ .

$$x = \sum_{i=1}^{d_e} e_i + \text{sigmoid}(\text{kernel}_e)W$$

This calculation of  $x$  applies the sigmoid function to the kernel to produce an interaction feature. A projection matrix  $W$  aligns its dimensions with  $x$ .

After tests on sequences of length 16, expanded sequences ran using a sparsity function in order to mimic using the original 16 word window plus 16 other words. The expanded sequences included 48, 64, 80, 96, and 112 words with each fitting into the total pruned size of 32.

$$\begin{aligned} k &= \text{pruned size}/2 \\ m &= k - 2 \\ y &= \text{norm}(\text{MHA}(Q, K_m, V_m))W \\ x &= x + \text{MHA}(Q, K_m, V_m) + y \\ x &= \text{concat}(x_{\text{Top}K(k, y, x)}, x_m) \end{aligned}$$

This calculation of  $x$  combines top  $k$  thresholding similar to (27) with the architecture of a single multi head attention step gating elements of the sequence from (5). It uses a context window plus a top  $k$  sparse slice similar to (28), but without compression or blocking, and with the extra MHA step before regular encoder layers. This sparse attention architecture rests on idea from (30), that blocks of text represent entities that sparse attention may gather. The structure of the feature vectors act as the blocking step, obviating much of the need to discover relationships by attention in blocks, making them safe to prune. Restricting the size of the key and value to the close context window also focuses the attention on that window and its relation to the larger text.

The MHA function takes the query of all  $n$  words and the key and value of a window set of  $m$  words. This example uses 48 words compressed into 32 words. Here, the window set is the 14 words surrounding the masked word. These 14 words join the other 16 words that the MHA function selects. A projection vector  $W$  reduces the normalized attention to a score for each word in the sequence. The normalized attention joins the input embeddings  $x$  as a secondary attention so that back propagation affects them all. The top 16 scores form the 16 other words, and a 32 word slice of the input passes to the rest of the model. This approach reduces the  $O(n^2)$  complexity to  $O(nm)$  by dropping the  $n$  for  $K$  and  $V$  and replacing it with  $m$ , the size of the window.

## Experiments

In all training, every other epoch writes a model until 40 training epochs. The results below are the maximum accuracy as a percent and its accompanying perplexity. Nearly all models encountered overfitting by 40 epochs. The baseline began overfitting at epoch 10.

There are 8 tests in Table 1, beginning with the baseline and then adding features. The feature counts below represent the size of the vector that holds them, with excesses in each subset truncated.

Adding the parser features improves accuracy by 79% and reduces perplexity by 63%. Drops in perplexity correlate with the rises in accuracy. The greatest contributions come from the features for dependencies. Aside from dependencies and dependency pos features, single features could not overcome the noise of random initializations. Therefore further ablative testing was abandoned, and they are bundled for testing.

Some classic measurements of improvement become moot or lose impact with feature vectors. The fertility of sub-word features does not increase from 1 when they combine in a

TABLE 1. Token vs Feature Vectors of Various Features

| Feature Count | Features                                           | Accuracy | Perplexity |
|---------------|----------------------------------------------------|----------|------------|
| 2             | Surface Form + Absolute Position (baseline)        | 13.43    | 2742.70    |
| 6             | + pos_ + Root + Suffixes                           | 16.05    | 1942.38    |
| 18            | + tag_ + lemma_ + ent.type_ + Sub-Suffixes + morph | 16.84    | 1971.65    |
| 19            | + Dependency (dep_)                                | 19.90    | 1725.92    |
| 28            | + Head Dep. + Rel. Positions + Rel. pos_ Positions | 21.14    | 1216.83    |
| 37            | + Gender + Coreferences + Inventory + Lemma POS    | 21.18    | 1112.06    |
| 39            | + Dependency Positions + Repeat Position           | 22.88    | 1193.45    |
| 40            | + Dependency pos                                   | 23.97    | 1021.22    |

feature vector. Fertility does not vary or respond to learning as in (10) and (2), and sub-word features do not need to account for statistical efficiency with sequence lengths. With word cosine similarity, the overlap of composable feature vectors overshadows their difference. Word relationships now carry features that appear on both sides. For a Word2Vec (14) example of  $X_{apple} - X_{apples} \approx X_{car} - X_{cars}$ , the left side expression  $X_{apple} - X_{apples}$  could be represented by  $[apple_{string}, apple_{lemma}, apple_{root}, noun, singular, Suffix_{NONE}] - [apples_{string}, apple_{lemma}, apple_{root}, noun, plural, Suffix_s]$ . There is a difference between *surface\_forms*, but more importantly a difference of *plural* + *Suffix\_s* and *singular* + *Suffix\_{NONE}*, as well as a similarity of  $apple_{lemma} + apple_{root} + noun$ .

These results show an aggregation of performance increases comparable with those in previous works concerning word prediction. Cross entropy loss improvements were 5-15% in (2) for next word prediction. Accuracy improvements were 1.7-9.7% in (3) for sentence completion (predict the last word in a sentence), 6% in (15) for sentence completion, 13-25% in (10) for last word prediction (predict the last word in a paragraph). Other experiments use BLEU scores and other classification or reasoning tasks, with most improvements in the range of 10-25%. All experiments following this point use the full 40 feature input.

There are 5 early fusion functions tested in Table 2.

TABLE 2. Early Fusion Functions

| Fusion Function       | Accuracy | Perplexity |
|-----------------------|----------|------------|
| Summation             | 23.97    | 1021.22    |
| Weight Vector         | 22.08    | 1617.95    |
| Concatenation         | 23.37    | 1223.91    |
| Gated Multimodal Unit | 21.81    | 1394.50    |
| Gated Fusion          | 22.95    | 1330.45    |

Summation had the highest single accuracy measurement. The gated multimodal unit function, which includes concatenation, and the concatenation function perform better on accuracy than the gated fusion and weighted vector function, which use summation. Since all features are learned and all relate to the words, multimodal fusion functions may not have anything truly different to fuse or gate. This may reflect that all the features fit together as different views of words. A more fitting measure of their interactions may be features based on activation functions whose values activate in the feed forward network. All experiments following this point use summation as their fusion function.

There are 5 interaction and activation functions tested in Table 3.

TABLE 3. Interaction and Activation Functions

| Interaction Function  | Accuracy | Perplexity |
|-----------------------|----------|------------|
| None                  | 23.97    | 1021.22    |
| Radial Basis Function | 23.43    | 1284.79    |
| Hyperbolic Tangent    | 23.89    | 1128.37    |
| Sigmoid               | 23.28    | 1249.45    |
| Cosine Similarity     | 24.27    | 1271.87    |

Cosine similarity showed an increase while the rest fell only slightly behind. Also, all interaction functions except one have better accuracy than all of the fusion functions except one. All experiments following this point use cosine similarity as their interaction function, which raises the feature count to 41.

There are 6 folding configurations tested in Table 4. They accumulate tags until only the root of the parse tree remains to be folded.

TABLE 4. Folding Configurations

| Folded Words                                              | Accuracy | Perplexity |
|-----------------------------------------------------------|----------|------------|
| None                                                      | 24.27    | 1271.87    |
| Punctuation + UNK + NUM + Interjection + Symbol + X       | 25.28    | 1037.03    |
| + Particle + Determiner + Pronoun + Auxilliary            | 26.29    | 995.42     |
| + Adverb + Adjective + Preposition + Conjunction          | 28.19    | 1019.29    |
| + Direct Object + Indirect Object + Object of Preposition | 27.22    | 1540.39    |
| + Compound + Verb                                         | 27.33    | 1003.92    |

Adding tags adds accuracy until the maximal configuration uses Punctuation + UNK + NUM + Interjection + Symbol + X + Particle + Determiner + Pronoun + Auxiliary + Adverb + Adjective + Preposition + Conjunction. As with word features, singular tags do not reliably confer improvements, so tags are grouped. The accuracy improves by 16% and perplexity by 20%. All experiments following this point use the maximal folding configuration, which raises the feature count to 44.

Testing sparsity begins with establishing benchmarks for dense attention without pruning. The below 10 tests with 2 models on 5 window lengths each in Table 5 cover increasing window lengths for models based on surface forms and for models with all of the features up to this point.

With models based on surface forms, increasing the window length in small data sets does not effect performance, and real improvement would probably require many more epochs and much more training data. With models based on parser output, expanding the window length improves accuracy by as much as 15% and perplexity by as much as 61%, but at the cost of a quadratic increase in complexity. The jumps up and down in accuracy and perplexity are an artifact of the noise in the small data set. There are no time trails here, but run time did limit the maximum length.

Again 10 tests of 2 models with 5 sequence lengths appear in Table 6. All models prune the sequence to 32 after the MHA function and pass that to the encoder layers.

With models based on surface forms, sparsity with pruning confers an improvement in accuracy by 33% and perplexity by 43%. With models based on parser output, improvements in accuracy by 15% and perplexity by 64% keep pace with dense attention, but with less



TABLE 5. Window Lengths

| Feature Number | Window Lengths | Accuracy | Perplexity |
|----------------|----------------|----------|------------|
| 2              | 48             | 13.34    | 2077.09    |
| 2              | 64             | 13.61    | 2108.68    |
| 2              | 80             | 13.42    | 2353.56    |
| 2              | 96             | 13.31    | 2613.89    |
| 2              | 112            | 12.11    | 3024.17    |
| 44             | 48             | 31.86    | 397.44     |
| 44             | 64             | 31.99    | 402.66     |
| 44             | 80             | 32.47    | 399.55     |
| 44             | 96             | 29.01    | 614.35     |
| 44             | 112            | 31.27    | 424.10     |

TABLE 6. Sparsity Configurations

| Feature Number | Window Lengths | Accuracy | Perplexity |
|----------------|----------------|----------|------------|
| 2              | 48             | 16.68    | 1658.15    |
| 2              | 64             | 17.91    | 1556.18    |
| 2              | 80             | 17.65    | 1572.53    |
| 2              | 96             | 14.21    | 2729.99    |
| 2              | 112            | 15.04    | 2447.69    |
| 44             | 48             | 30.85    | 489.01     |
| 44             | 64             | 30.05    | 492.60     |
| 44             | 80             | 30.42    | 587.66     |
| 44             | 96             | 32.56    | 358.26     |
| 44             | 112            | 31.42    | 796.62     |

complexity. The total accuracy difference of models of feature vectors with sparse attention over those of surface forms without sparse attention is 143% with a perplexity difference of 87%.

## Conclusions

Additional inputs start a model on a greater foundation of semantic content and of syntactic form for representing relationships of language. A transformer model based on the surface forms might eventually contain similar implicit syntactic and semantic relationships as latent relationships from the data set. Explicitly moving a large segment of this information into the input raises model performance and generalization as measured by masked word prediction accuracy and perplexity. The extra dimensions of the input do not interact as independent modes and might be better described as interacting parts, views, or relationships. Integrating parser output allows novel means of augmenting and combining words, here through additional tags and folding. Sparse attention with pruning can improve models from surface forms and from parser output. The limits of sparse attention with feature vectors and of the relationship between  $n$  and  $m$  remain unexplored here at high data volumes, where sparse attention consistently falls below dense attention. Their impact here at low data volumes raise the floor of transformer performance by 143% in accuracy and by 87% in perplexity.

## References

- (1) J. Arevalo et al. *Gated Multimodal Units for Information Fusion*. 2017. arXiv: 1702.01992 (stat.ML). URL: <https://arxiv.org/abs/1702.01992>.
- (2) E. Asgari, Y. El Kheir, and M. Javaheri. “MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies”. In: (Feb. 2025). DOI: 10.48550/arXiv.2502.00894.
- (3) J. Bian, B. Gao, and T.-Y. Liu. “Knowledge-Powered Deep Learning for Word Embedding”. In: *Machine Learning and Knowledge Discovery in Databases*. Nancy France: Springer-Verlag, 2022, pp. 132–148. ISBN: 978-3-662-44847-2. DOI: 10.1007/978-3-662-44848-9\_9. URL: [https://doi.org/10.1007/978-3-662-44848-9\\_9](https://doi.org/10.1007/978-3-662-44848-9_9).
- (4) T. Bolukbasi et al. *Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings*. 2016. arXiv: 1607.06520 (cs.CL). URL: <https://arxiv.org/abs/1607.06520>.
- (5) R. Child et al. *Generating Long Sequences with Sparse Transformers*. 2019. arXiv: 1904.10509 (cs.LG). URL: <https://arxiv.org/abs/1904.10509>.
- (6) L. Choshen et al. *BabyLM Turns 4: Call for Papers for the 2026 BabyLM Workshop*. 2026. arXiv: 2602.20092 (cs.CL). URL: <https://arxiv.org/abs/2602.20092>.
- (7) S. Egea Gómez, E. McGill, and H. Saggon. “Syntax-aware Transformers for Neural Machine Translation: The Case of Text to Sign Gloss Translation”. In: *Proceedings of the 14th Workshop on Building and Using Comparable Corpora (BUCC 2021)*. Ed. by R. Rapp, S. Sharoff, and P. Zweigenbaum. Online (Virtual Mode): INCOMA Ltd., Sept. 2021, pp. 18–27. URL: <https://aclanthology.org/2021.bucc-1.4/>.
- (8) C. Han and S. Jalagam. *Metadata Extraction Leveraging Large Language Models*. 2025. arXiv: 2510.19334 (stat.ML). URL: <https://arxiv.org/abs/2510.19334>.
- (9) M. Honnibal et al. *spaCy: Industrial-strength Natural Language Processing in Python*. 2020. DOI: 10.5281/zenodo.1212303. URL: <https://doi.org/10.5281/zenodo.1212303>.
- (10) H. Jabbar. *MorphPiece : A Linguistic Tokenizer for Large Language Models*. 2024. arXiv: 2307.07262 (cs.CL). URL: <https://arxiv.org/abs/2307.07262>.
- (11) K. Lang. *20 Newsgroups Dataset*. Accessed: 2026-03-11. 1995. URL: <http://people.csail.mit.edu/jrennie/20Newsgroups/>.
- (12) Y. Liu and Y. Hou. *Syntax-Aware Complex-Valued Neural Machine Translation*. 2024. arXiv: 2307.08586 (cs.CL). URL: <https://arxiv.org/abs/2307.08586>.
- (13) M.-C. de Marneffe et al. “Universal Stanford dependencies: A cross-linguistic typology”. In: *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC’14)*. Ed. by N. Calzolari et al. Reykjavik, Iceland: European Language Resources Association (ELRA), May 2014, pp. 4585–4592. URL: <https://aclanthology.org/L14-1045/>.
- (14) T. Mikolov, W.-t. Yih, and G. Zweig. “Linguistic Regularities in Continuous Space Word Representations”. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by L. Vanderwende, H. Daumé III, and K. Kirchhoff. Atlanta, Georgia: Association for Computational Linguistics, June 2013, pp. 746–751. URL: <https://aclanthology.org/N13-1090/>.
- (15) T. Mikolov et al. “Efficient Estimation of Word Representations in Vector Space”. In: *Proceedings of Workshop at ICLR 2013* (Jan. 2013).

- (16) T. Mikolov et al. “Distributed Representations of Words and Phrases and their Compositionality”. In: *CoRR* abs/1310.4546 (2013). arXiv: 1310.4546. URL: <http://arxiv.org/abs/1310.4546>.
- (17) J. Nivre et al. “Universal Dependencies v2: An Evergrowing Multilingual Treebank Collection”. In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. Marseille, France: European Language Resources Association, May 2020, pp. 4034–4043. URL: <https://aclanthology.org>.
- (18) A. Razzhigaev et al. *LLM-Microscope: Uncovering the Hidden Role of Punctuation in Context Memory of Transformers*. 2025. arXiv: 2502.15007 (cs.CL). URL: <https://arxiv.org/abs/2502.15007>.
- (19) B. Santorini. *Part-of-speech tagging guidelines for the Penn Treebank Project*. Tech. rep. MS-CIS-90-47. Philadelphia, PA, USA: Department of Computer and Information Science, University of Pennsylvania, 1990. URL: <ftp://ftp.cis.upenn.edu/pub/treebank/doc/tagguide.ps.gz>.
- (20) P. Shaw, J. Uszkoreit, and A. Vaswani. *Self-Attention with Relative Position Representations*. 2018. arXiv: 1803.02155 (cs.CL). URL: <https://arxiv.org/abs/1803.02155>.
- (21) N. Stylianou and I. Vlahavas. “E.T.: Entity-Transformers. Coreference augmented Neural Language Model for richer mention representations via Entity-Transformer blocks”. In: *Proceedings of the Third Workshop on Computational Models of Reference, Anaphora and Coreference*. Ed. by M. Ogrodniczuk et al. Barcelona, Spain (online): Association for Computational Linguistics, Dec. 2020, pp. 1–10. URL: <https://aclanthology.org/2020.crac-1.1/>.
- (22) Z. Su et al. “CGT: A Clause Graph Transformer Structure for aspect-based sentiment analysis”. In: *Data & Knowledge Engineering* 153 (2024), p. 102332. ISSN: 0169-023X. DOI: <https://doi.org/10.1016/j.datak.2024.102332>. URL: <https://www.sciencedirect.com/science/article/pii/S0169023X24000569>.
- (23) L. Tunstall, L. von Werra, and T. Wolf. *Natural Language Processing with Transformers*. O’Reilly, 2022. ISBN: 978-1-098-13679-6.
- (24) A. Vaswani et al. “Attention Is All You Need”. In: *CoRR* abs/1706.03762 (2017). arXiv: 1706.03762. URL: <http://arxiv.org/abs/1706.03762>.
- (25) X. Wang et al. “Multi-Modal Generative AI: Multi-Modal LLMs, Diffusions, and the Unification”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 36.4 (Apr. 2026), pp. 5621–5641. ISSN: 1558-2205. DOI: 10.1109/tcsvt.2025.3635224. URL: <http://dx.doi.org/10.1109/TCSVT.2025.3635224>.
- (26) T. Wenzel, G. Santin, and B. Haasdonk. “Analysis of structured deep kernel networks”. In: *Journal of Computational and Applied Mathematics* 476 (Apr. 2026), p. 116975. ISSN: 0377-0427. DOI: 10.1016/j.cam.2025.116975. URL: <http://dx.doi.org/10.1016/j.cam.2025.116975>.
- (27) C. You et al. “Spark Transformer: Reactivating Sparsity in Transformer FFN and Attention”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025. URL: <https://openreview.net/forum?id=o4zN34ahEK>.
- (28) J. Yuan et al. *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. 2025. arXiv: 2502.11089 (cs.CL). URL: <https://arxiv.org/abs/2502.11089>.
- (29) X. Zhang, J. Zhao, and Y. LeCun. “Character-level Convolutional Networks for Text Classification”. In: *Advances in Neural Information Processing Systems* 28. 2015,

- pp. 649–657. URL: [http://www.di.unipi.it/~gulli/AG\\_corpus\\_of\\_news\\_articles.html](http://www.di.unipi.it/~gulli/AG_corpus_of_news_articles.html).
- (30) H. Zhao et al. *Structured-Sparse Attention for Entity Tracking with Subquadratic Sequence Complexity*. 2026. arXiv: 2605.22476 (cs.LG). URL: <https://arxiv.org/abs/2605.22476>.
- (31) H. Zhao et al. “Do Transformers Parse while Predicting the Masked Word?” In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by H. Bouamor, J. Pino, and K. Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 16513–16542. DOI: 10.18653/v1/2023.emnlp-main.1029. URL: <https://aclanthology.org/2023.emnlp-main.1029/>.
- Email address: maconnor34@gmail.com*